In each of these scenarios, you'll need to configure a TLS certificate for your application so Kestrel can receive HTTPS traffic. In section 28.2 you'll see the easiest way to get started with HTTPS when developing locally, using the ASP.NET Core development certificate.

28.2 Using the ASP.NET Core HTTPS development certificates

Working with HTTPS certificates is easier than it used to be, but unfortunately it can still be a confusing topic, especially if you're a newcomer to the web. In this section you'll learn how the .NET software development kit (SDK), Visual Studio, and IIS Express try to improve this experience by handling a lot of the grunt work for you, and what to do when things go wrong.

The first time you run a `dotnet` command using the .NET SDK, the SDK installs an HTTPS development certificate on your machine. Any ASP.NET Core application you create using the default templates (or for which you don't explicitly configure certificates) will use this development certificate to handle HTTPS traffic. However, the development certificate is not trusted by default. If you access a site that's using an untrusted certificate, you'll get a browser warning, as shown in figure 28.4.

A brief primer on certificates and signing

HTTPS uses *public key cryptography* as part of the data-encryption process. This uses two keys: a public key that anyone can see and a private key that only your server can see. Anything encrypted with the public key can be decrypted only with the private key. That way, a browser